

「智慧笔迹」NLP 新闻分类学习赛 —— 课设详解

项目：26 秋小学期课设（源自天池大赛「零基础入门 NLP——新闻文本分类」）
仓库：<https://github.com/siyiliang41-collab/NLP-News-Classification-Learning-Contest>

0. 一句话概括

这是一个 **14 类中文新闻文本多分类** 任务，数据被**字符级匿名化**（每篇文章变成一串空格分隔的数字 token），评测指标是 **macro F1**。项目从「传统机器学习基线」一路做到「深度学习 + 预训练模型 + 多模型融合」，把分数从 **0.8463** 拉升到 **0.9332**，并用一套可复用的工程结构（统一配置、分数台账、可视化、答辩演示界面）支撑了完整的研究报告。

1. 赛题与任务定义

1.1 数据构成

数据	规模	字段
训练集 <code>train_set.csv</code>	200,000 条	<code>label + text</code> (<code>\t</code> 分隔)
测试集 A <code>test_a.csv</code>	50,000 条	只有 <code>text</code> (无 <code>label</code>)
测试集 B <code>test_b.csv</code>	50,000 条	后期放出，本课设未使用

14 个类别（官方固定编号，见 `src/config.py`）：

科技0 股票1 体育2 娱乐3 时政4 社会5 教育6
财经7 家居8 游戏9 房产10 时尚11 彩票12 星座13

1.2 数据的特殊性：字符级匿名化

看一条真实训练样本（`train_set.csv` 第 2 行）：

```
label=2（体育）
text = 2967 6758 339 2021 1854 3731 4109 ...
```

每篇新闻都被打散成一串数字 token。这是赛题的防作弊设计——如果文本是明文，选手可以直接去搜新闻原文、甚至人工标注测试集。匿名化之后：

1. **中文分词失效**：没有真实汉字，jieba 等分词器无从下手，只能按空格切（每个 token 天然是一个「词」）。
2. **预训练词向量失效**：开源的「中文 Word2Vec / BERT 词表」都是基于真实汉字训练的，数字 token 根本不在词表里，无法直接加载。
3. **但 token 映射是一致的**：同一篇语料里，同一个真实字/词 → 永远映射到同一个数字。所以 token 的**共现关系**、**n-gram 模式**、**出现频率**这些统计结构被完整保留了。

关键推论（这是整个项目建模的出发点）：匿名化只是「把词典换了」，并没有抹掉文本的统计规律。因此这条路仍然成立——**在训练集上重新学习 token 级别的表示（自训练词向量），再让模型捕捉 token 的局部/全局组合模式**。README 的技术路线正是基于这个判断展开的。

2. EDA：用数据回答「难在哪」

`src/eda.py` 产出了三张图 + 一份报告（`docs/EDA分析报告.md`）。EDA 的价值在于：它直接决定了后续所有建模决策。三个核心发现：

2.1 发现一：类别严重不均衡（43 : 1）

- 最多的是**科技**（38,918 条），最少的是**星座**（908 条），相差约 **43 倍**。
- 但评测指标是 **macro F1**，每个类别 F1 直接平均、**不看样本量**。这意味着星座、彩票、时尚这些小类，和科技、股票**同等重要**——预测错 100 条科技，跟预测错 100 条星座，扣分一样多。

由此引出两个动作：

- 交叉验证用 `StratifiedKFold`（分层抽样），保证每折类别比例一致（否则小类可能在某折里直接消失）；
- 专门做了一组**损失函数消融实验**（CE / 加权 CE / Focal Loss），去验证「类别不均衡到底是不是主要瓶颈」（答案很有戏剧性，见 §6）。

2.2 发现二：文本长度长尾分布

- 平均 907 token, 中位数 676, 最长 12,211 token, ≤ 600 的样本约占 44.5%。
- **决策**: 截断长度 `MAX_SEQ_LEN = 600` (`src/config.py`)。理由: 新闻的**标题和导语**通常就承载了主题, 截断保留前部信息损失不大, 却能大幅节省显存和计算。

2.3 发现三: 存在超高频 token

- 第一名 `#3750` 出现 **748 万次**, 远超第二名 (492 万次)。
- 很可能是匿名化前的**标点符号或高频停用字** (如「的」「了」「,」「。」「」)。

3. 评测指标 macro F1 的原理

这是理解「为什么这样设计」的前提, 必须讲清楚。

二分类的 F1 是精确率 P 和召回率 R 的调和平均:

$$F1 = \frac{2 \times P \times R}{P + R}, \quad P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

多分类时, 先对 14 个类别**逐个**算 F1 (把该类当正例、其余当负例, 即 one-vs-rest), 再取算术平均:

$$\text{macro F1} = \frac{1}{C} \sum_{i=1}^C F1_i$$

对比之下, `micro F1` 是把所有类别的 TP/FP/FN 加总后算一个全局 F1 (等价于准确率, 会被大类主导), `weighted F1` 是按样本量加权。赛题选 macro, 就是为了**强制大家重视小类**——这也正是本项目反复做类别不均衡分析的原因。

项目里所有模型统一用 sklearn 的 `f1_score(..., average="macro")` 计算 (`src/common.py`), 保证分数口径一致、可横向比较。

4. 技术路线总览: 逐级上分的逻辑

README 里写的路线:

```
TF-IDF + LR (baseline) → Word2Vec + TextCNN  
→ BiLSTM + Attention → BERT 微调 → 多模型融合
```

对应的分数 (来自 `docs/scores.csv`, 可视化脚本 `src/plot_scores.py`):

阶段	模型	macro F1	备注
① 基线	TF-IDF + LR	0.8463 ± 0.0038	5 折 CV
② 深度·CNN	Word2Vec + TextCNN	0.9183 ± 0.0024	5 折 CV, skip-gram
③ 深度·RNN	Word2Vec + BiLSTM + Attn	0.9241	holdout
④ 预训练	BERT 微调	0.9240	holdout
⑤ 融合	硬投票 (3 模型)	0.9332	BERT+TextCNN+TFIDF

这五个台阶的设计意图非常清晰，每一级都在解决上一级的短板：

- ① 到 ②：从「词袋 (bag-of-words)」升级到「能学习词向量 + 捕捉局部 n-gram」的神经网络，突破线性模型的表达能力上限；
- ② 到 ③：从「只看局部窗口 (CNN)」升级到「能建模序列长程依赖 (RNN) + 自动分配注意力权重」；
- ③ 到 ④：从「从零自训练词向量」升级到「利用大规模预训练模型 + 迁移学习」；
- ④ 到 ⑤：单模型已到平台期，靠「不同结构模型犯错不同」的**集成**再榨出 0.9 个点。

5. 各级算法原理详解

5.1 ① TF-IDF + 逻辑回归 (SGD 求解) —— 基线

文件： `src/baseline_tfidf_lr.py`

TF-IDF 原理

TF-IDF 是文本分类最经典的「把文本变成向量」的方法。核心思想：**一个 token 对一篇文章越重要，它在这篇里出现得越频繁 (TF)，同时在整个语料里出现得越少 (IDF)。**

- **TF (词频)**： `tf(t,d)` = token `t` 在文档 `d` 中的次数。
- **IDF (逆文档频率)**： `idf(t) = log(N / (1 + df(t)))`，其中 `N` 是总文档数，`df(t)` 是包含 `t` 的文档数。出现得越普遍，权重越低。
- 最终 `tfidf(t,d) = tf(t,d) × idf(t)`。

工程上的四个精妙设置 (`src/baseline_tfidf_lr.py`)

```
TfidfVectorizer(  
    token_pattern=r"\d+",          # 只匹配数字 token，用 C 级正则切分，远快于 Python split  
    ngram_range=(1, 2),          # 1-gram + 2-gram，捕捉相邻 token 的组合
```

```
max_features=50000,      # 限制特征维度，防过拟合 + 控内存
min_df=3,                # 出现少于 3 篇的 token 丢弃（去噪）
sublinear_tf=True,       # tf 取 1+log(tf)，抑制高频词的线性膨胀
)
```

几个点值得注意：

- `token_pattern=r"\d+"`：因为数据是「空格分隔的数字 token」，直接让 sklearn 用正则匹配数字串，比默认 `token_pattern` 更贴合，且走 C 层、速度快一个量级。
- `ngram_range=(1,2)`：匿名化后单个数字 token 没有语义，**相邻两个 token 的组合**（类比「两个字组成的词」）能提供更多判别信息，是提分关键之一。
- `sublinear_tf`：对 TF 做对数压缩，防止某个超高频 token（如 #3750）在向量里主导一切。

逻辑回归（Softmax 回归）原理

线性模型给每个类别一个得分 $z_k = w_k \cdot x + b_k$ ，再用 softmax 转成概率：

$$P(y = k|x) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

训练目标是最大化正确类别的对数概率（等价最小化交叉熵）。

为什么用 SGDClassifier 而不是 LogisticRegression

这是代码注释里明确写的一个工程取舍：

- `LogisticRegression(liblinear)` 求解器做多分类时，是 **one-vs-rest**：要为 14 个类别训练 **14 个二分类器**，在 20 万样本上非常慢。
- `SGDClassifier(loss="log_loss")` 用的是**随机梯度下降**，直接做**原生多分类**（一个 softmax 输出 14 个类），`loss="log_loss"` 让它等价于逻辑回归的目标，但速度快一个量级、精度相当。

训练与评估流程

用 `StratifiedKFold(n_splits=5)` 做 5 折交叉验证：每折在训练子集上 `fit_transform`（注意是**折内拟合** vectorizer，防止信息泄漏），在验证子集上 `transform` 后预测，算 macro F1，最后取 5 折均值作为模型的「诚实成绩」。

结果：**0.8463 ± 0.0038**。标准差很小，说明模型稳定。这是后续所有模型要超越的基准线。

5.2 ② Word2Vec + TextCNN —— 让神经网络学会「看文本」

文件： `src/textcnn.py`

这一步解决的是基线的两个短板：词袋模型**丢失了词序信息**，且**特征不能随任务学习**。方案分两段：

第一段：自训练 Word2Vec 词向量

因为数据是匿名的，**没有现成的中文词向量可用**，只能在训练集上自训练。原理

(`src/common.py`)：

- **CBOW (sg=0)**：用上下文 token 预测中心 token，训练快、适合本场景；
- **Skip-gram (sg=1)**：用中心 token 预测上下文，更慢但更精细，**textcnn.py 最终用了 skip-gram**。
- 把每条文本 `split()` 后截断到 600，作为「句子」喂给 gensim，学出每个 token 的 128 维稠密向量。

Word2Vec 的核心思想是**分布式假设**：「出现在相似上下文中的 token，语义相近」。虽然 token 是数字，但它们在语料里的上下文分布仍然蕴含信息，学出来的向量能让「功能相似的 token」在向量空间里靠近。

学习完后把词向量映射成**嵌入矩阵** (`src/common.py`)：(词表大小, 128)，未命中的 token 用小的随机数初始化，PAD 位置置零。这张矩阵作为 TextCNN 的**词嵌入层初始值**。

第二段：TextCNN 模型

TextCNN 是 Kim (2014) 提出的经典文本分类 CNN。核心思想：**用不同尺寸的卷积核去扫字符/词的 n-gram，自动提取局部特征**。

模型结构 (`src/common.py`)：

1. **嵌入层**： `nn.Embedding(vocab_size, 128)`，用上面的 Word2Vec 矩阵初始化，且 `requires_grad=True` **允许微调**。
2. **多卷积核**： `filter_sizes = [2, 3, 4]`，每种 100 个卷积核 (`num_filters=100`)。卷积核高度 = 2/3/4 (对应 2-gram、3-gram、4-gram 的窗口)，宽度 = 128 (整个词向量维度)。
 - 对输入 (batch, 1, seq, 128) 做二维卷积，核尺寸 (fs, 128) 的卷积相当于「沿序列滑动，每次看 fs 个连续 token 的完整向量」。
3. **ReLU + 全局最大化**：每个卷积核输出一个特征图，取**最大值**——直觉是「这个 n-gram 模式在文中出现的最强信号」。
4. **拼接 + Dropout + 全连接**：三种核的池化结果拼成 (batch, 3×100)，过 Dropout(0.5) 防过拟合，最后线性层输出 14 类 logits。

结果： **0.9183 ± 0.0024** （5 折 CV，比基线提升 **+7.2 个百分点**）。这是全项目单模型里幅度最大的一次跃升，说明「词向量 + 局部卷积」确实抓住了匿名文本的判别模式。

5.3 ③ Word2Vec + BiLSTM + Attention —— 建模长程依赖

文件：`src/bilstm_attn.py`，模型在 `src/common.py`

CNN 的局限是**感受野固定**——一个 4-gram 卷积核只看 4 个 token，无法建模「第 3 个词和第 500 个词之间」的长距离关系。新闻分类里，主题往往散布在全文各处，于是引入 RNN。

LSTM 原理（门控记忆）

LSTM 通过三个「门」控制信息流动，解决普通 RNN 的梯度消失问题：

- **遗忘门** f_t ：决定丢掉多少旧记忆；
- **输入门** i_t ：决定写入多少新信息；
- **输出门** o_t ：决定输出多少。

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C), \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t)$$

BiLSTM（双向）

单向 LSTM 只能看到「过去」，但文本分类里「未来」同样重要。双向 LSTM 跑两条 LSTM（正向 + 反向），每个时间步把两个方向的隐状态拼接，得到 `(batch, seq, 2×128)` 的输出——每个位置都同时包含左右上下文。

Attention 注意力机制

LSTM 输出是**每个时间步一个向量**，但做分类只需要**一个**句子向量。最简单的做法是取最后一个时间步，但那会丢掉大部分信息。Attention 的做法是：**让模型自己学「哪些时间步重要」**。

```
scores = torch.softmax(self.attn(out), dim=1) # (batch, seq, 1) 每个位置的权重
context = torch.sum(out * scores, dim=1)      # 加权求和 → (batch, 2*hidden)
```

- `self.attn` 是一个线性层，把每个时间步的隐向量映射成一个标量分数；
- `softmax` 归一化成概率（所有权重和为 1）；

- 加权求和得到「上下文向量」，其中关键 token 贡献大、填充/噪音 token 贡献小。

这相当于给模型一种能力：**自动聚焦到对分类最有信息量的 token 上**，而不用人工指定。

结果：**0.9241** (holdout, 比 TextCNN 再升 +0.6)。注意这里为省时用的是「留出 10% 验证集」而非 5 折 CV (--holdout)，所以和 0.9183 不是严格同口径，但趋势可信。

5.4 ④ BERT 微调 —— 预训练 + 迁移学习

文件：src/bert.py

出发点与预期

BERT 是基于 Transformer 的预训练模型，用海量中文语料做过「掩码语言模型」和「下一句预测」两个自监督任务。**理论上**它已经学会了丰富的语言知识，下游分类只需微调。

但这里有个**诚实的洞察**（写在 src/bert.py 的注释里）：

文本是字符级匿名化的数字 token，用空格连接后喂给 bert-base-chinese；BERT 的预训练语义帮不上太多，主要靠微调学习 token 共现模式。

也就是说：bert-base-chinese 的词表是真实汉字，而输入是数字串，tokenizer 会把数字切成一个个数字 token，**预训练学到的「语义」在这里基本用不上**，模型实际是在「从随机初始化近似的位置重新学习 token 模式」。所以 BERT 的分数 **0.9240** 只是和 BiLSTM (0.9241) **打平**，没有碾压——这恰恰是匿名化数据导致的预期结果，也是报告中很有说服力的一个分析点。

实现细节（几个工程亮点）

1. **成本控制**：用 holdout (10% 验证集) 而非 5 折 CV (VAL_RATIO=0.1)，因为 5 折 = 5 倍训练时间。默认还提供 --subset 子集模式（默认 50000 条）。
2. **学习率调度**：AdamW + get_linear_schedule_with_warmup，先线性 warm-up 再线性衰减——这是 BERT 微调的标准做法，避免训练初期大梯度破坏预训练权重。
3. **Checkpoint 续跑**：save_checkpoint / load_checkpoint，应对云 GPU 实例被抢占/意外中断，续跑不浪费已训的 epoch。
4. **只保存最佳模型**：每个 epoch 在验证集上评估，只有 F1 创新高才保存。

BERT 的 Transformer 原理（简要）

BERT 的核心是 Transformer 的 Encoder 堆叠，关键机制是**自注意力 (Self-Attention)**：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

每个 token 都会和句子里的**所有** token 计算注意力权重，从而直接建模任意两个 token 之间的依赖——相比 LSTM 的「逐步传递」，这是全局、并行的。配合多头注意力、位置编码、前馈网络和残差连接，BERT 能学到深层的上下文表示。

5.5 ⑤ 多模型融合（硬投票）—— 集成学习的最后一击

文件： `src/ensemble.py`

为什么融合能涨分

集成学习的核心直觉：**不同结构的模型，犯的错不一样**。CNN 擅长局部 n-gram，RNN 擅长长程依赖，BERT 是全局注意力，TF-IDF+LR 是词袋统计。当三个模型「投票」时，只要不是三个同时错同一条样本，多数票就能纠正个别模型的错误。

硬投票（Majority Vote）原理

由于本赛题提交文件**只有 label 一列、没有 softmax 概率**，所以只能用硬投票：

```
# 每条样本，取所有模型预测的众数
cnt = Counter(votes)
result.append(cnt.most_common(1)[0][0])
```

- 3 个模型对同一条样本投票，取票数最多的类别；
- **平票规则**（代码注释如实记录了）：`Counter.most_common(1)` 平票时返回「第一个出现」的类别，即按模型列表顺序靠前者胜出。当 3 个模型各投一票时，会退化成「无条件取第 1 个模型」——这在该样本上就没发挥融合作用。注释里诚实说明了这一点。
- 代码里还实现了 `weighted_vote` 作为备选，可按模型可信度加权打破平票。

融合结果

0.9332，融合了 BERT + TextCNN + TF-IDF-LR 三个模型，是项目最高分。

代码还打印了模型间的一致率，用来观察「融合能带来多少互补空间」。

6. 亮点实验：损失函数消融（CE vs 加权 vs Focal）

这是全项目**最有研究价值**的一组实验，文件：`src/textcnn_loss.py`，可视化
`src/plot_loss_ablation.py`。

6.1 背景动机

EDA 发现了「43 倍类别不均衡」，教科书式的应对是改损失函数。于是项目在 TextCNN 上对比三种损失：

损失	原理	macro F1
CE (交叉熵)	标准多分类损失， <code>-log p_正确类</code>	0.9042
加权 CE	给少数类更大权重 <code>1/类别样本数</code>	0.8750
Focal Loss	加权 + 聚焦难样本	0.8886

6.2 三种损失的原理

交叉熵 CE：

$$\mathcal{L}_{CE} = -\log(p_y)$$

加权交叉熵 (`src/common.py`)：给每个类一个权重 `w_k = 1 / 样本数_k`（再归一化到均值 1），样本越少的类权重越大：

$$\mathcal{L}_{weighted} = -w_y \log(p_y)$$

Focal Loss (`src/common.py`)：在 CE 基础上乘一个调制因子，让模型「聚焦难分类的样本」：

$$\mathcal{L}_{focal} = -\alpha_y(1 - p_y)^\gamma \log(p_y)$$

- 当样本**已经分类得很好**（`p_y` 接近 1），`(1-p_y)^\gamma` 趋近 0，损失被大幅压低——模型不再把精力浪费在「已经会的简单样本」上；
- 当样本**分类困难**（`p_y` 小），调制因子接近 1，损失保持，逼模型重点学习难样本。
- 这里 `gamma=2`，且叠加了类别权重 `alpha`。

6.3 反直觉的结论（核心洞察）

三类损失下的**逐类 F1** 显示：

- CE** 在「大但容易混淆」的新闻域（**社会 0.8444、时政 0.8826、房产 0.9079**）更强；
- Focal** 在「小但主题鲜明」的类（**星座 0.9565、彩票 0.9315、时尚 0.8920**）更强；
- 但**加权 / Focal 的整体 macro F1 反而低于 CE**。

脚本注释里给出了精辟的结论：

本数据集瓶颈是「类间混淆」而非「类别不均衡」。

换句话说：小类样本虽少，但**主题鲜明、本身就好分**（星座、彩票的 F1 其实很高）；真正拖低 macro F1 的是**社会、时政、房产这些大类的互相混淆**。加权/Focal 把火力对准小类，却牺牲了大类的区分度，结果「捡了芝麻丢了西瓜」。

这组实验的价值不在分数，而在它否证了一个想当然的假设——报告里能讲出「我们验证过类别不均衡优化手段、并解释了为什么最终不用它」，这比直接调参上分更能体现科研素养。

注意：这组是 `--holdout`, `epochs=3`, CBOW 的快速配置，所以 TextCNN 的 CE 只有 0.9042，和正式版（5 折、`epochs=5`、skip-gram）的 0.9183 不是同口径，但三种损失之间是**公平对比**的。

7. 工程实现：一套可复用的脚手架

这个项目不只是「几个脚本」，而是一套结构清晰的工程。值得逐一提：

7.1 统一配置 `src/config.py`

- **数据路径自动定位**：依次探测环境变量、本地 Windows 路径、项目内 `data/`、阿里云 PAI DSW、AutoDL 等位置，本地/云端**免改路径**。
- **类别映射唯一来源**：`LABEL_MAP` 和 `ID2LABEL`，所有脚本从这里 import，避免 14 个类别的硬编码散落各处、出现不一致。
- **统一超参**：`MAX_SEQ_LEN`、`N_FOLDS`、`SEED` 集中管理。

7.2 公共组件 `src/common.py`

把多个脚本重复的代码抽出来复用：`Vocab`（词表）、`NewsDataset`（数据集）、`TextCNN` / `BiLSTM_Attention`（模型）、`FocalLoss` / `compute_class_weights`（损失）、`train_word2vec` / `build_embedding_matrix`（词向量）、`train_one_epoch` / `evaluate` / `predict`（训练评估循环）。`textcnn_loss.py` 和 `bilstm_attn.py` 都从这里 import，避免了重复造轮子。

7.3 分数台账 `src/scoreboard.py`

每个实验跑完，用 `record()` 把（模型名，变体，`cv_f1`，std，备注）追加到 `docs/scores.csv`。这保证了**所有实验分数有据可查、可追溯**，也直接喂给可视化脚本。

7.4 性能迭代可视化 `src/plot_scores.py`

读 scores.csv，取每个模型的最好成绩，画**柱状图**（对比）+ **折线图**（上升趋势）。脚本注释明确写了这是「设计报告评分表的硬性要求——性能迭代过程的可视化」。

7.5 答辩演示界面 `src/app.py`

用 Gradio 搭了一个交互界面，供答辩现场演示：

- 输入匿名 token 文本 → 输出预测类别 + 14 类概率条形图；
- 「随机取一条测试样本」按钮，展示真实效果（因为明文中文无法直接预测，演示以随机取样为主）。

8. 已有成果总结与解读

8.1 完整成绩单

模型	macro F1	提升
TF-IDF + LR（基线）	0.8463	—
Word2Vec + TextCNN	0.9183	+7.2
Word2Vec + BiLSTM + Attn	0.9241	+0.6
BERT 微调	0.9240	≈持平
多模型硬投票	0.9332	+0.9（总 +8.7）

8.2 三个可写进报告的结论

1. **从词袋到神经网络的跃迁是最大收益**：+7.2 个点，验证了「自训练词向量 + 局部卷积」对匿名文本有效；
2. **BERT 在匿名化数据上优势消失**：与 BiLSTM 打平（0.9240 vs 0.9241），说明预训练语义失效后，BERT 退化为「更贵的从零学习」，这是匿名化数据的重要特征；
3. **类别不均衡不是主要瓶颈，类间混淆才是**：加权/Focal 反而降分，真正的短板是「社会/时政/房产」等大类互相混淆（损失消融实验直接证明）。

8.3 可继续改进的方向（供报告「未来工作」）

- **更强的集成**：当前硬投票受「无 softmax 概率」限制，若保留各模型的**概率输出**做加权/Stacking，有望再涨分；

- **针对长文本**：600 截断只覆盖 44.5% 样本，可做「头尾截断」或「分块池化」保留尾部信息；
 - **类别混淆的专门处理**：既然瓶颈是「社会/时政/房产」混淆，可对这些易混淆类别做**二阶段分类或层次分类**；
 - **BERT 的替代用法**：把匿名 token 当作「伪字符」重新预训练（领域自适应预训练），而非直接用中文 BERT。
-

9. 总结

这个课设完整走完了「**理解数据 (EDA)** → **定义指标** → **逐级建模 (4 种范式)** → **消融验证假设** → **集成冲刺** → **工程化交付**」的研究闭环。它的价值体现在三个层面：

1. **算法层面**：覆盖了文本分类的四大主流范式——词袋+线性、CNN、RNN+Attention、预训练 Transformer，以及集成学习和损失函数设计；
2. **研究层面**：不是「无脑堆模型」，而是用 EDA 定位难点、用消融实验**证伪**了「类别不均衡是瓶颈」的直觉假设，得出「类间混淆才是关键」的可信结论；
3. **工程层面**：统一配置、公共组件、分数台账、自动可视化、Gradio 演示，构成了一个可复用、可答辩、可扩展的完整项目。